crossfit: A Graph-Based Cross-Fitting Engine in R

by Etienne Peyrot and François Petit

Abstract Semiparametric procedures such as double/debiased machine learning (DML) use cross-fitting to evaluate targets with out-of-sample nuisance predictions. The crossfit package provides a general R engine in which users define a target functional and the nuisance models it requires. It executes an explicit schedule over folds and repeated data partitions, returning either a scalar estimate or a cross-fitted predictor function. The package also supports nuisance dependency graphs, alternative allocations of training folds across nuisance components, and comparisons of several methods under shared splits. It reuses identical nuisance fits across methods and provides method-specific failure controls, which are useful in simulation studies and method development. Deterministic schedules and validation checks make the resulting cross-fitting procedure auditable. The package is available on CRAN and is openly developed on GitHub under GPL-3.

1 Introduction

1.1 Cross-fitting and DML in brief

Suppose the parameter of interest is a low-dimensional target θ_0, while its estimation also requires one or more nuisance functions collected in η_0. Examples of nuisance functions include an outcome regression, a propensity score, or a data-dependent preprocessing rule. Flexible methods can reduce modeling restrictions for these functions, but evaluating a target with nuisance predictions fitted on the same observations can transmit overfitting bias to the target estimate.

Sample splitting separates these two operations: one part of the data is used to fit the nuisance functions, and a disjoint part is used to evaluate the target. Their roles are rotated over a K-fold partition. In the simplest case, this proceeds as follows:

1. Divide the observations into K folds.
2. Fit the nuisance functions without one fold.
3. Predict the nuisance functions and evaluate the target on the held-out fold.
4. Repeat the fitting and evaluation steps until every fold has served as the held-out fold, then combine the fold-specific target estimates.

Each observation therefore contributes to target evaluation through nuisance predictions obtained from fits that excluded that observation.

This approach has a long history in semiparametric statistics (Bickel, 1982; Schick, 1986; Zheng and van der Laan, 2011; Newey and Robins, 2018). It is a central component of double/debiased machine learning (DML) (Chernozhukov et al., 2018) and orthogonal statistical learning (Foster and Syrgkanis, 2023). In these approaches, the target is commonly represented by a score $\psi(W; \theta, \eta)$. Neyman orthogonality makes the expected score locally insensitive to small perturbations of the nuisance functions, while cross-fitting controls the dependence between nuisance estimation and score evaluation. Together, these properties permit flexible nuisance estimation while retaining valid inference under suitable regularity and rate conditions.

1.2 Why a general cross-fitting engine is useful

The **crossfit** package is designed for settings where the cross-fitting scheme is itself part of the method to implement or study. The researcher defines the target and the nuisance quantities it requires, while the package organizes their fitting and out-of-sample evaluation.

This separation is particularly useful for newly developed semiparametric estimators, for which the cross-fitting procedure would otherwise have to be implemented specifically for each estimator.

This focus differs from that of packages such as **DoubleML**, which provide complete estimation and inference procedures for established DML model classes (Bach et al., 2024). **crossfit** instead exposes the cross-fitting construction itself. It can therefore accommodate estimators with several nuisance quantities, including settings where one nuisance depends on predictions from another or where different nuisance quantities must be fitted on different subsets of the available training data. These features are useful when the statistical properties of an estimator depend on how nuisance functions are trained relative to one another, for example, when nuisance estimates should be based on separate training samples to limit dependence between their estimation errors.

The package is also intended for methodological comparisons and simulation studies. Competing estimators can be evaluated using the same random partitions, alternative ways of allocating training data among their nuisance functions can be compared without rewriting the estimator, and identical nuisance fits shared across methods can be computed only once. These capabilities, together with the need for reproducible cross-fitting schedules across large simulation studies and benchmarks of causal estimators, motivated the development of the package; preliminary versions were used in Bouvier et al. (2024).

For an estimator already covered by a **DoubleML** model class, **DoubleML** provides a direct route from nuisance estimation to inference. When the goal is instead to implement a researcher-defined cross-fitted estimator, control how its nuisance quantities are trained, or compare alternative cross-fitting designs under a common schedule, **crossfit** provides the corresponding general-purpose engine. The following example first illustrates its basic use before these additional controls are introduced.

2 A first cross-fitting analysis

Consider estimating the prediction error of a flexible regression method. With a two-fold split, the regression is fitted on the first fold and evaluated on the second. The roles of the folds are then reversed, and the two held-out errors are averaged. This simple procedure is a common form of cross-fitted estimation.

The example below follows the basic procedure used throughout the package: define a nuisance model, define the target, create a method specification, and run the schedule. The simulated outcome depends nonlinearly on `x`, and the nuisance regression is fitted with a random forest.

```
set.seed(123)
n <- 300
sim_data <- data.frame(x = runif(n, -2, 2))
sim_data$y <- sin(pi * sim_data$x) + rnorm(n, sd = 0.3)

nuis_y <- create_nuisance(
  fit = function(data) {
    ranger::ranger(y ~ x, data = data, num.trees = 100, seed = 1)
  },
  predict = function(model, data) {
    as.numeric(predict(model, data = data)$predictions)
  },
  train_fold = 1L
)

target_mse <- function(data, nuis_y, ...) mean((data$y - nuis_y)^2)

m <- create_method(
```

```
  target = target_mse,
  list_nuisance = list(nuis_y = nuis_y),
  folds = 2L,
  repeats = 1L,
  eval_fold = 1L,
  mode = "estimate",
  fold_allocation = "overlap",
  aggregate_panels = mean_estimate,
  aggregate_repeats = mean_estimate
)

res <- crossfit(sim_data, m, seed = 123)
res$estimate

#> [1] 0.12092
```

The target is evaluated twice, once for each held-out fold. The function supplied through `aggregate_panels` averages these two mean squared errors. The same separation between evaluation data and nuisance fitting data is used when cross-fitting nuisance predictions for DML, although the target is then an orthogonal score rather than a prediction error.

A complete DML application to the partially linear regression model is provided in Appendix DML for the partially linear regression model.

3 Package concepts and design

The first analysis used the package's central objects. This section defines those objects and the schedule they produce.

3.1 User-facing abstractions

Two functions define the package's main objects.

- **`create_nuisance()`** defines a nuisance node through a `fit()` function and a `predict()` function. Any fitting method that provides these two operations can be used, and `train_fold` specifies how many folds train the node.
- **`create_method()`** combines a target functional with its named nuisances and the settings for data splitting, fold allocation, aggregation, and failure handling. A target is an R function with explicit nuisance arguments, such as `target(data, nuis_y, nuis_m)`.

Two output modes are supported. In `mode = "estimate"`, the target returns a numeric value. In `mode = "predict"`, it returns a prediction function for new data. Appendix Prediction mode gives a complete example.

3.2 Folds, panels, evaluation windows, and repetitions

The main notions introduced in this section, including folds, evaluation windows, and panels, are illustrated in Figure 1. In `mode = "estimate"`, the data are first partitioned into `folds = K` subsets. The target is evaluated on an **evaluation window**, which is a set of one or more consecutive folds. The argument `eval_fold` gives the width of this window. Nuisance predictions supplied to the target are produced from fits whose training observations exclude the current evaluation window. Prediction mode instead uses `eval_fold = 0`.

A **panel** is one position of the evaluation window in the cyclic schedule. With `K = 5` and `eval_fold = 2`, the five panels evaluate on folds $(1,2)$, $(2,3)$, $(3,4)$, $(4,5)$, and $(5,1)$. The

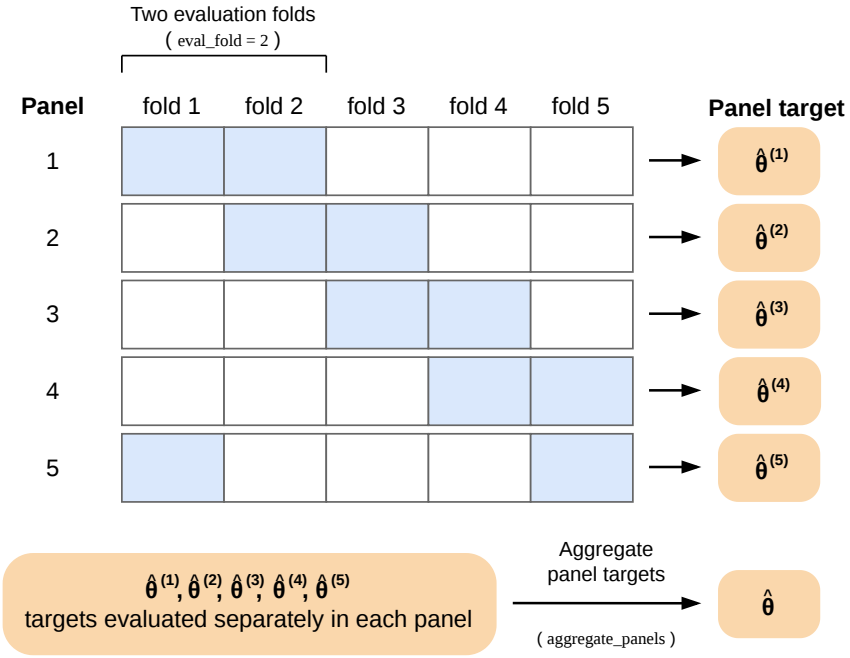


Figure 1: Schedule for five-fold cross-fitting with an *evaluation window* width of two folds (`eval_fold` = 2). Each row is a panel, and the highlighted *evaluation window* advances cyclically by one fold. The target is evaluated separately in each panel, producing the panel targets shown with superscripts indexing panels. The function supplied through `aggregate_panels` combines these panel targets. Unhighlighted folds lie outside the current *evaluation window*; their use for nuisance training depends on `train_fold` and `fold_allocation`.

window advances by one fold between panels and wraps around after fold 5. The target is evaluated separately in each panel, giving the panel targets $\widehat{\theta}^{(1)}, \ldots, \widehat{\theta}^{(5)}$; the superscript indexes panels in Figure 1.

One **repetition** consists of one K-fold partition, the complete cycle of K panels, and aggregation of the corresponding panel targets. The split remains fixed throughout that cycle. The `repeats` argument specifies how many independently generated partitions are processed in this way.

3.3 Aggregation over panels and repetitions

Within each repetition, the target is evaluated separately in each of the K panels. The resulting panel-specific target values are then combined through `aggregate_panels` to obtain one repetition-specific estimate. When `repeats > 1`, the data are partitioned again, the complete panel schedule is rerun, and another estimate is obtained. The function supplied through `aggregate_repeats` then combines these repetition-specific estimates into the final value. The two aggregation functions are specified separately and need not be identical.

Using `repeats = 1` corresponds to ordinary K-fold cross-fitting and requires only one random partition. Repeating the splitting and aggregating the resulting estimates can reduce sensitivity to a particular random partition; Jacob (2020) illustrates this finite-sample benefit for heterogeneous treatment-effect estimators. Repeated splitting requires fitting the nuisance functions again, however, and no universal number of repetitions is recommended.

The package provides `mean_estimate()` and `median_estimate()` for numeric targets, and `mean_predictor()` and `median_predictor()` for prediction functions; custom aggregation functions may also be supplied. In `mode = "predict"`, the panel-specific and repetition-specific prediction functions are combined pointwise when called on new data.

3.4 Reading the returned object

For one method, `crossfit()` returns the final value in the `estimate` component. The list `results$values` contains the panel-aggregated value from each successful repetition before `aggregate_repeats` is applied, while `results$errors` records panel or aggregation errors. The remaining components report completed repetitions, fold requirements, the validated method, and the execution plan. Single-method results are not indexed by method name.

3.5 Nuisance dependency graph and validation

Nuisance components form the nodes of a directed acyclic graph (DAG). A dependency arises when a nuisance's `fit()` or `predict()` function requires predictions from another nuisance component. The same representation covers data-dependent preprocessing, such as imputation, normalization, calibration, or feature construction. Before execution, the package validates nuisance graphs and method specifications:

- **Argument coverage checks:** required arguments in nuisance `fit()` and `predict()` functions must have corresponding nuisance mappings.
- **Cycle detection:** cyclic dependency graphs are rejected early with informative errors.
- **Target-nuisance consistency:** the target function's required nuisance arguments must be provided by the method specification.
- **Input sanitization:** fold constraints, aggregation functions, failure controls, and mode-specific requirements are checked before execution.

These checks make invalid specifications explicit before the cross-fitting schedule begins.

3.6 Fold allocation modes

All fold allocation modes in **crossfit** satisfy the same central requirement: in every mode, the target evaluation fold/window is disjoint from all nuisance training folds used to generate the nuisance predictions passed to the target. This is the usual out-of-sample guarantee targeted by standard DML cross-fitting (Chernozhukov et al., 2018; Newey and Robins, 2018).

Allocation of training folds across nuisances. Modes differ only in how training folds are assigned across nuisance instances inside a panel, that is, whether different nuisance models or branches of a nuisance dependency graph may reuse the same training folds.

This distinction matters because, beyond the leading orthogonality-driven cancellation, many doubly robust/orthogonal estimators have a remainder containing products of nuisance estimation errors. If nuisance errors are correlated because the nuisances were trained on the same data, this "nonlinear" term can behave poorly in finite samples. Several papers therefore consider strengthened splitting geometries that deliberately separate training samples across nuisance components. The terminology varies across the literature ("double cross-fitting" (Newey and Robins, 2018; McClean et al., 2026), "three-way cross-fitting" (Fisher and Fisher, 2023), "triple cross-fit" (Ke et al., 2026), and "multiway cross-fitting" (Chiang et al., 2022)), but the common principle is to train the nuisance components that interact in the remainder term on separate subsamples.

The three allocation modes. Figure 2 shows the same method, represented by a triangle-shaped nuisance DAG, under the three modes for `K = 5`. It shows the panel in which the target is evaluated on fold 1 (blue); the other panels follow by cyclic shifts modulo `K`.

The modes differ as follows:

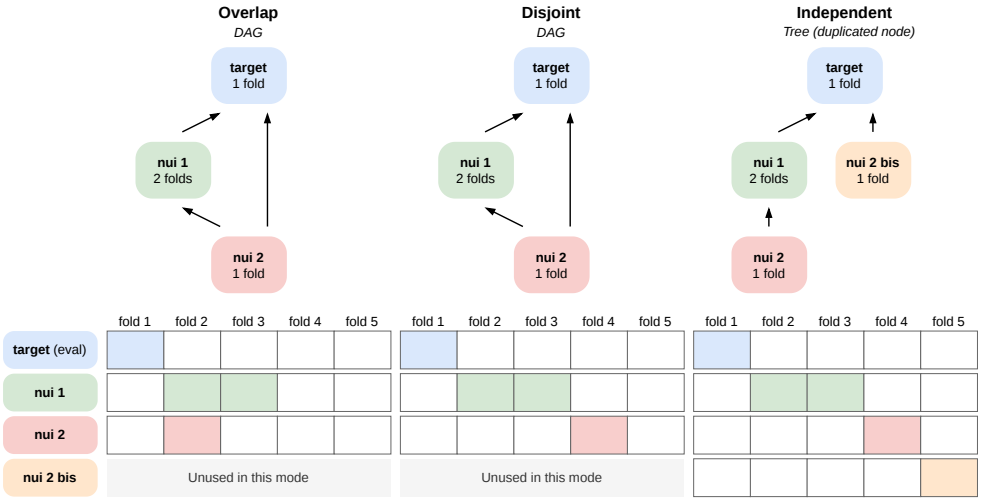


Figure 2: Fold allocation modes in crossfit. The same method is executed under three fold allocation modes. `target` denotes the user-defined target functional (the final quantity computed), and `nui 1`/`nui 2` denote two required nuisance models; the arrow `nui 2` → `nui 1` indicates a nuisance-of-nuisance dependency. Top: nuisance dependency structure and node-specific training widths (`train_fold` = number of folds used to train each nuisance). Bottom: for `K = 5`, the panel where the target is evaluated on fold 1 (blue) is shown. Colored cells indicate the training folds used to fit each nuisance instance in that panel. `nui 2 bis` is a duplicated copy of `nui 2` used only in Independence mode.

- **`"overlap"`** corresponds to the original, widely used cross-fitting geometry in DML. It enforces the out-of-sample constraint for the evaluation window, but allows different nuisances to use overlapping, often identical, training folds within a panel (Chernozhukov et al., 2018; Newey and Robins, 2018).
- **`"disjoint"`** implements the central idea behind "double", "triple", and "three-way" designs in their simplest form. Within a panel, it coordinates training-fold assignment across nuisance instances to avoid fold reuse when possible, so nuisance models that jointly contribute to a second-order remainder term can be trained on different data (McClean et al., 2026; Fisher and Fisher, 2023; Ke et al., 2026). Shared nodes are not duplicated, so dependence can remain through an upstream fit that is used in several places.
- **`"independence"`** applies a stronger separation. If a nuisance node is used in several branches, the engine duplicates that node by expanding the graph into a tree, allowing the branches to use disjoint training folds (Chiang et al., 2022). This mode can require a larger `K` and more nuisance fits, but provides the strongest separation when a method uses the same nuisance in several places.

Practical trade-offs. Overlap mode corresponds to the default geometry in classical DML and is often a natural baseline (Chernozhukov et al., 2018). Disjoint and independence modes control how much training data is shared across nuisance models. This connects to an active research question: when do stronger splitting geometries improve finite-sample performance by limiting dependence-driven second-order bias terms (McClean et al., 2026; Fisher and Fisher, 2023; Ke et al., 2026; Chiang et al., 2022; Zivich and Breskin, 2021)? The three modes permit these geometries to be compared without changing the target or nuisance definitions.

3.7 Computation, reuse, and failure handling

Nuisance fitting generally dominates the cost of simulation-heavy cross-fitting studies. The engine evaluates nuisance nodes in dependency order and reuses a fitted object only when the complete nuisance specification and its training-fold window are identical. Reuse is limited to one repetition because each repetition uses a new data partition.

When several methods are compared, `crossfit_multi()` applies one schedule to all named method specifications in the call. The methods therefore use the same data partitions, and an identical nuisance request is fitted only once. Reuse is limited to structurally shared instances and nuisance nodes required in several places. Appendix Benchmarking multiple methods gives a complete example.

The `"disjoint"` and `"independence"` allocation modes can introduce more nuisance instances than `"overlap"`. The cyclic panel schedule can nevertheless request the same instance more than once, so the number of graph nodes does not translate directly into the number of fits. Stronger allocation modes may also require a larger `K`, in which case the schedule contains more panels.

Errors are recorded separately for each method. Through `failure_control`, a method specifies whether a panel error ends the current repetition, whether a previous failure of an identical shared fit prunes later work in that repetition, and how many failed repetitions are permitted. Other methods in the same `crossfit_multi()` call continue according to their own controls.

Users may also supply a deterministic `fold_split` function, enabling exact reproduction of fold schedules across machines and code revisions. The engine itself is sequential; higher-level parallelization is discussed in Section Discussion, limitations, and reuse.

4 Technical details and quality control

4.1 Automated unit and integration testing

The automated test suite uses **testthat** (edition 3). Unit tests cover specification helpers (`create_nuisance()`, `create_method()`), aggregation utilities, and validation logic. Integration tests cover `crossfit()` and `crossfit_multi()` in estimation and prediction modes.

4.2 Cross-fitting invariants: no-leakage testing

Automated invariant tests verify that each nuisance prediction supplied to a target comes from a fit that excludes the corresponding evaluation fold. This check covers the `"overlap"`, `"disjoint"`, and `"independence"` allocation modes.

4.3 Robustness: failure handling

Panel and aggregation errors are recorded separately for each method. By default, a panel error ends the current repetition. With `fail_repetition_on_error = FALSE`, the remaining panels are evaluated and failed panels are passed as `NA_real_` to `aggregate_panels`. The `max_failed_repetitions` setting controls when a repeatedly failing method is disabled, while other methods continue. The optional `prune_on_shared_fit_failure` setting skips a method when a structurally identical nuisance fit has already failed in the same repetition.

4.4 Continuous integration

GitHub Actions runs `R CMD check` on every push and pull request. Checks cover macOS and Windows with R release, and Ubuntu Linux with R release and R-devel. This matrix reduces operating-system-specific regressions and tests compatibility with the next R release.

5 Discussion, limitations, and reuse

5.1 Limitations and scope

The package supplies cross-fitting rather than ready-made estimators. Its main limitations are:

- **No built-in estimators:** the package does not provide causal estimators, orthogonal scores, or standard-error formulas. Users define targets and inference procedures, and use the package to obtain cross-fitted nuisance predictions or target evaluations.
- **Fold-geometry constraints:** stronger allocation modes such as `"disjoint"` and especially `"independence"` may require larger numbers of folds `K` (or may be infeasible for a given nuisance graph and node-specific `train_fold` settings). They can also increase the number of nuisance instances to fit, trading statistical dependence reduction for additional computation.
- **No internal parallelization:** the engine does not parallelize over folds, panels, or repetitions. Independent simulation replications or method grids can be parallelized outside a package call with standard R tools.
- **User-defined code remains part of the analysis:** the schedule enforces out-of-sample fold separation, but nuisance functions should not use global state, retain data between calls, or access variables that should be unavailable at prediction time.

5.2 Reuse potential

The package can be used whenever estimation of a low-dimensional target requires out-of-sample nuisance predictions, including predictions from data-dependent preprocessing such as imputation. Examples include DML estimators for causal parameters, partially linear models, orthogonal-score-based targets, and procedures that require several dependent nuisance components. Because nuisances are defined through user-provided `fit()` and `predict()` functions, a wide range of R fitting methods can be used without imposing a particular model class.

Prediction functions. In addition to scalar target estimation, `mode = "predict"` constructs a function from cross-fitted nuisance predictions. This supports prediction on new data and methods that combine several cross-fitted prediction functions. Appendix Prediction mode gives a complete example.

Empirical methodology reuse. The package is well suited to simulation and benchmarking studies: users can vary `K`, repeated splitting, node-specific training widths, and fold allocation modes to study finite-sample behavior under controlled schedules, including dependence-reduction geometries motivated by double, triple, and three-way cross-fitting (McClean et al., 2026; Fisher and Fisher, 2023; Ke et al., 2026; Chiang et al., 2022). One repetition is the cheaper baseline; additional repetitions can be used when sensitivity to a random partition is itself relevant. The deterministic `fold_split` interface supports reproducible experiments, including grouped or time-based splits when appropriate. As described in Section Computation, reuse, and failure handling, `crossfit_multi()` applies identical splits to competing methods and can reuse shared nuisance fits.

5.3 Extending the software

The core engine can be extended in several directions:

- **New targets:** implement new target functionals by writing an R function that consumes `data` and named nuisance predictions.
- **New nuisances:** wrap any fitting method with a fit/predict interface as a nuisance node; express sequences of dependent fits through nuisance dependencies.
- **Custom fold splitting:** supply a custom `fold_split` function to define folds deterministically or under complex sampling schemes.

- **Aggregation and diagnostics:** provide custom functions that aggregate target values after panel evaluation and, in `mode = "predict"`, combine predictor functions pointwise.

5.4 Support and contributions

Development and support are managed through the GitHub repository. Users can file bug reports and feature requests through the issue tracker (`/issues`). Contributions are welcomed via pull requests; continuous integration provides automated feedback on portability and correctness across the supported OS/R matrix.

Availability

Operating system. Any operating system supported by R. Automated checks run on Ubuntu Linux, Windows, and macOS via GitHub Actions.

Programming language. R (>= 4.1.0).

System requirements. None beyond a standard R installation.

Dependencies. Imports: **stats**, **utils**.

Source code and archive. The **crossfit** package is available on CRAN: https://CRAN.R-project.org/package=crossfit. Development occurs on GitHub under GPL-3: https://github.com/EtiennePeyrot/crossfit-R.

Installation.

```
# Install the released version from CRAN
install.packages("crossfit")

# Install the development version from GitHub
install.packages("remotes")
remotes::install_github("EtiennePeyrot/crossfit-R", build_vignettes = TRUE)
```

Acknowledgements

The authors thank the open-source R community for foundational tooling (including **testthat**, **knitr**, and **rmarkdown**) that supports reliable package development and documentation.

Funding

Etienne Peyrot acknowledges support from the Université Paris Cité. François Petit acknowledges support from the French Agence Nationale de la Recherche through the project reference ANR-22-CPJ1-0047-01.

Competing interests

The authors declare that they have no competing interests.

Appendix

5.5 Recipe: DML for the partially linear regression model

This appendix gives a complete, self-contained example of a double/debiased machine learning (DML) estimator for the partially linear regression (PLR) model using the `pension`

dataset from the **hdm** package (Chernozhukov et al., 2016). The user defines the nuisance models and target functional, and the package applies the out-of-sample schedule. The example follows the classical 401(k) application in Chernozhukov et al. (2018).

Model and estimand

In the PLR model, the effect of a treatment variable D on an outcome Y is assumed to be linear with coefficient θ_0, while the effect of covariates X is allowed to be nonparametric:

$$\begin{aligned} Y &= D\theta_0 + g_0(X) + U, \quad \mathbb{E}[U \mid X, D] = 0, \\ D &= m_0(X) + V, \qquad\qquad \mathbb{E}[V \mid X] = 0. \end{aligned} \tag{1}$$

When D is binary, $m_0(X) = \mathbb{E}[D \mid X] = \mathbb{P}(D = 1 \mid X)$ is the propensity score. DML estimates θ_0 by (i) estimating nuisance functions g_0 and m_0 with flexible methods, (ii) residualizing Y and D, and (iii) regressing the residualized outcome on the residualized treatment using out-of-sample nuisance predictions.

Data: the `pension` 401(k) dataset

We use `pension` from **hdm**. Following common practice, we take:

- outcome Y: `tw` (total wealth),
- treatment D: `p401` (participation in a 401(k), binary),
- controls X: a standard subset of demographic and income indicators (listed below).

Implementation with crossfit

The DML "partialling-out" estimator can be written as:

$$\begin{aligned} \widehat{\theta} &= \frac{\sum_{i \in \mathcal{I}_{\text{eval}}} \widetilde{D}_i \widetilde{Y}_i}{\sum_{i \in \mathcal{I}_{\text{eval}}} \widetilde{D}_i^2}, \\ \widetilde{Y}_i &= Y_i - \widehat{g}(X_i), \qquad \widetilde{D}_i = D_i - \widehat{m}(X_i), \end{aligned} \tag{2}$$

where $\widehat{g}$ and $\widehat{m}$ are trained on folds disjoint from the evaluation set $\mathcal{I}_{\text{eval}}$. The implementation defines two nuisance models and the target functional.

Complete code This appendix uses the suggested packages **hdm**, **ranger**, and **glmnet** for the examples (Chernozhukov et al., 2016; Wright and Ziegler, 2017; Friedman et al., 2010; Tay et al., 2023); they are not required dependencies of **crossfit**.

```
library(hdm)
library(ranger)
library(crossfit)

data(pension)

y <- pension$tw
d <- pension$p401

xvar <- c(
  "i2","i3","i4","i5","i6","i7",
  "a2","a3","a4","a5",
  "fsize","hs","smcol","col",
  "marr","twoearn","db","pira","hown"
)
```

```
X <- pension[ , xvar]
dat <- data.frame(y = y, d = d, X)
dat <- na.omit(dat)

nuis_g <- create_nuisance(
  fit = function(data) {
    ranger::ranger(y ~ . - d, data = data, num.trees = 20, seed = 1)
  },
  predict = function(model, data) {
    as.numeric(predict(model, data = data)$predictions)
  }
)

nuis_m_rf <- create_nuisance(
  fit = function(data) {
    data$d <- factor(data$d, levels = c(0, 1))   # ensure classification
    ranger::ranger(d ~ . - y, data = data, probability = TRUE,
                   num.trees = 20, seed = 1)
  },
  predict = function(model, data) {
    as.numeric(predict(model, data = data)$predictions[ , "1"])
  }
)

target_plr <- function(data, nuis_g, nuis_m) {
  y_tilde <- data$y - nuis_g
  d_tilde <- data$d - nuis_m
  sum(d_tilde * y_tilde) / sum(d_tilde^2)
}

m_plr <- create_method(
  target = target_plr,
  list_nuisance = list(nuis_g = nuis_g, nuis_m = nuis_m_rf),
  folds = 5,
  repeats = 2,
  eval_fold = 1L,
  mode = "estimate",
  fold_allocation = "overlap",
  aggregate_panels = mean_estimate,
  aggregate_repeats = median_estimate
)

res_plr <- crossfit(dat, m_plr)
res_plr$estimate

#> [1] 11177.27
```

This recipe uses `repeats = 2` to demonstrate aggregation across two independently generated partitions with `median_estimate`. Setting `repeats = 1` gives the standard, less expensive baseline described in Section Aggregation over panels and repetitions.

Fold allocation modes

The same estimator can be run under the `"overlap"`, `"disjoint"`, and `"independence"` fold allocation modes without changing its nuisance or target definitions.

```
m_plr_overlap <- m_plr

m_plr_disjoint <- create_method(
  target = target_plr,
  list_nuisance = list(nuis_g = nuis_g, nuis_m = nuis_m_rf),
  folds = 5,
  repeats = 2,
  eval_fold = 1L,
  mode = "estimate",
  fold_allocation = "disjoint",
  aggregate_panels = mean_estimate,
  aggregate_repeats = median_estimate
)

m_plr_indep <- create_method(
  target = target_plr,
  list_nuisance = list(nuis_g = nuis_g, nuis_m = nuis_m_rf),
  folds = 5,
  repeats = 2,
  eval_fold = 1L,
  mode = "estimate",
  fold_allocation = "independence",
  aggregate_panels = mean_estimate,
  aggregate_repeats = median_estimate
)

res_modes <- crossfit_multi(
  data = dat,
  methods = list(
    overlap = m_plr_overlap,
    disjoint = m_plr_disjoint,
    independence = m_plr_indep
  )
)

res_modes$estimates

#> $overlap
#> [1] 11800.45
#>
#> $disjoint
#> [1] 10141.89
#>
#> $independence
#> [1] 10141.89
```

Notes. (i) We use random forests from **ranger** to illustrate nonparametric nuisance estimation. Each forest uses `num.trees = 20` to keep the examples fast; applications would generally use more trees and tune the model. The multi-method recipe additionally compares the random-forest propensity model with an ℓ_1-penalized logistic regression from **glmnet**. The same target functional can be used with any fitting method that provides compatible `fit()` and `predict()` functions. (ii) `"disjoint"` and especially `"independence"` may require larger K for some nuisance graphs and training-fold widths; infeasible specifications are rejected during validation.

5.6 Recipe: Nuisance dependency graphs (nuisance-of-nuisance)

Some DML procedures estimate nuisance functions in sequence. For example, an outcome model may use an estimated propensity score as an input. The package represents these dependencies as a directed acyclic graph (DAG) and evaluates the nuisances in dependency order. A nuisance declares a dependency by listing the upstream nuisance prediction as a named argument of its `fit()` or `predict()` function. Argument coverage is checked, and cyclic dependencies are rejected.

Propensity-augmented outcome regression inside PLR

The code below defines the propensity nuisance `nuis_m`, then defines an outcome nuisance `nuis_g_ps` whose `fit()` and `predict()` functions take `nuis_m` as an additional argument. This induces the edge `nuis_m -> nuis_g_ps` in the nuisance graph. The target remains the PLR partialling-out estimator.

```
nuis_g_ps <- create_nuisance(
  fit = function(data, nuis_m) {
    data$nuis_m <- nuis_m
    ranger::ranger(y ~ nuis_m + . - d, data = data, num.trees = 20, seed = 1)
  },
  predict = function(model, data, nuis_m) {
    data$nuis_m <- nuis_m
    as.numeric(predict(model, data = data)$predictions)
  }
)
m_plr_dag <- create_method(
  target = target_plr,
  list_nuisance = list(nuis_g = nuis_g_ps, nuis_m = nuis_m_rf),
  folds = 5,
  repeats = 2,
  eval_fold = 1L,
  mode = "estimate",
  fold_allocation = "overlap",
  aggregate_panels = mean_estimate,
  aggregate_repeats = median_estimate
)

res_plr_dag <- crossfit(dat, m_plr_dag)
res_plr_dag$estimate

#> [1] 12147.27
```

Remark. Nuisance dependencies can represent sequential model fitting, calibration, imputation, stacking, or other intermediate quantities that must be computed out of sample.

5.7 Recipe: Prediction mode (returning a cross-fitted predictor function)

In `mode = "predict"`, the target returns a prediction function rather than a scalar, and the engine combines the panel-specific functions into one function for new data. This mode applies when the quantity of interest is a reusable prediction rule.

Cross-fitted propensity score predictor

The following recipe returns a cross-fitted predictor function for the propensity score $m_0(X) = \mathbb{E}[D \mid X]$ on the `pension` dataset.

```
target_ps <- function(data, nuis_m, ...) nuis_m

m_ps <- create_method(
  target = target_ps,
  list_nuisance = list(nuis_m = nuis_m_rf),
  folds = 5,
  repeats = 2,
  eval_fold = 0L,
  mode = "predict",
  fold_allocation = "overlap",
  aggregate_panels = mean_predictor,
  aggregate_repeats = median_predictor
)

res_ps <- crossfit(dat, m_ps)

pscore_fun <- res_ps$estimate
pscore_fun(dat[1:5, ])

#> [1] 0.1962586 0.2485667 0.4256164 0.2734839 0.3261687
```

Remark. The same construction applies to other methods that form predictions from cross-fitted nuisance functions.

5.8 Recipe: Benchmarking multiple methods with `crossfit_multi()`

Section Computation, reuse, and failure handling introduces `crossfit_multi()` in the context of shared schedules and nuisance-fit reuse. This recipe compares two nuisance specifications under the same schedule.

Two PLR estimators with different propensity models

The two PLR estimators share the outcome nuisance `nuis_g` and the target functional `target_plr`. They differ in the propensity model: one uses a random forest fitted with **ranger** (Wright and Ziegler, 2017), and the other uses an ℓ_1-penalized logistic regression fitted with **glmnet** (Friedman et al., 2010; Tay et al., 2023).

```
nuis_m_lasso <- create_nuisance(
  fit = function(data) {
    x <- model.matrix(d ~ . - y - 1, data = data)
    y <- data$d
    glmnet::cv.glmnet(x, y, family = "binomial", alpha = 1)
  },
  predict = function(model, data) {
    x <- model.matrix(d ~ . - y - 1, data = data)
    as.numeric(predict(model, newx = x, s = "lambda.1se", type = "response"))
  }
)

m_plr_rf <- create_method(
  target = target_plr,
  list_nuisance = list(nuis_g = nuis_g, nuis_m = nuis_m_rf),
  folds = 5,
  repeats = 2,
  eval_fold = 1L,
  mode = "estimate",
```

```
  fold_allocation = "overlap",
  aggregate_panels = mean_estimate,
  aggregate_repeats = median_estimate
)

m_plr_lasso <- create_method(
  target = target_plr,
  list_nuisance = list(nuis_g = nuis_g, nuis_m = nuis_m_lasso),
  folds = 5,
  repeats = 2,
  eval_fold = 1L,
  mode = "estimate",
  fold_allocation = "overlap",
  aggregate_panels = mean_estimate,
  aggregate_repeats = median_estimate
)

res_cmp <- crossfit_multi(
  data = dat,
  methods = list(plr_rf = m_plr_rf, plr_lasso = m_plr_lasso)
)

res_cmp$estimates

#> $plr_rf
#> [1] 10755.66
#>
#> $plr_lasso
#> [1] 10565.04
```

Remark. The method names index the `estimates` component, while repetition-level values and error diagnostics are stored in `per_method`. Because both methods use the same fold schedule and share `nuis_g`, the engine can reuse that fitted nuisance where the complete specification and training-fold window are identical.

References

P. Bach, M. S. Kurz, V. Chernozhukov, M. Spindler, and S. Klaassen. DoubleML: An object-oriented implementation of double machine learning in R. *Journal of Statistical Software*, 108(3):1–56, 2024. doi: 10.18637/jss.v108.i03. [p2]

P. J. Bickel. On adaptive estimation. *The Annals of Statistics*, 10(3):647–671, 1982. doi: 10.1214/aos/1176345863. [p1]

F. Bouvier, E. Peyrot, A. Balendran, C. Ségalas, I. Roberts, F. Petit, and R. Porcher. Do machine learning methods lead to similar individualized treatment rules? A comparison study on real data. *Statistics in Medicine*, 43(11):2043–2061, 2024. doi: 10.1002/sim.10059. [p2]

V. Chernozhukov, C. Hansen, and M. Spindler. hdm: High-dimensional metrics. *The R Journal*, 8(2):185–199, 2016. doi: 10.32614/RJ-2016-040. [p10]

V. Chernozhukov, D. Chetverikov, M. Demirer, E. Duflo, C. Hansen, W. Newey, and J. Robins. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal*, 21(1):C1–C68, 2018. doi: 10.1111/ectj.12097. [p1, 5, 6, 10]

H. D. Chiang, K. Kato, Y. Ma, and Y. Sasaki. Multiway cluster robust double/debiased machine learning. *Journal of Business & Economic Statistics*, 40(3):1046–1056, 2022. doi: 10.1080/07350015.2021.1895815. [p5, 6, 8]

A. Fisher and V. Fisher. Three-way cross-fitting and pseudo-outcome regression for estimation of conditional effects and other linear functionals, 2023. URL https://arxiv.org/abs/2306.07230. [p5, 6, 8]

D. J. Foster and V. Syrgkanis. Orthogonal statistical learning. *The Annals of Statistics*, 51(3): 879–908, 2023. doi: 10.1214/23-AOS2258. [p1]

J. Friedman, T. Hastie, and R. Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010. doi: 10.18637/jss.v033.i01. [p10, 14]

D. Jacob. Cross-fitting and averaging for machine learning estimation of heterogeneous treatment effects, 2020. URL https://arxiv.org/abs/2007.02852. [p4]

D. Ke, X. Zhou, Q. Yang, and X. Song. Doubly robust triple cross-fit estimation for causal inference with imaging data. *Statistics in Biosciences*, 18(3):776–796, 2026. doi: 10.1007/s12561-024-09458-1. [p5, 6, 8]

A. McClean, S. Balakrishnan, E. H. Kennedy, and L. Wasserman. Double cross-fit doubly robust estimators: Beyond series regression. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 88(4):1469–1491, 2026. doi: 10.1093/jrsssb/qkag057. [p5, 6, 8]

W. K. Newey and J. M. Robins. Cross-fitting and fast remainder rates for semiparametric estimation, 2018. URL https://arxiv.org/abs/1801.09138. [p1, 5, 6]

A. Schick. On asymptotically efficient estimation in semiparametric models. *The Annals of Statistics*, 14(3):1139–1151, 1986. doi: 10.1214/aos/1176350055. [p1]

J. K. Tay, B. Narasimhan, and T. Hastie. Elastic net regularization paths for all generalized linear models. *Journal of Statistical Software*, 106(1):1–31, 2023. doi: 10.18637/jss.v106.i01. [p10, 14]

M. N. Wright and A. Ziegler. ranger: A fast implementation of random forests for high dimensional data in C++ and R. *Journal of Statistical Software*, 77(1):1–17, 2017. doi: 10.18637/jss.v077.i01. [p10, 14]

W. Zheng and M. J. van der Laan. Cross-validated targeted minimum-loss-based estimation. In M. J. van der Laan and S. Rose, editors, *Targeted Learning: Causal Inference for Observational and Experimental Data*, Springer Series in Statistics, pages 459–474. Springer, New York, NY, 2011. doi: 10.1007/978-1-4419-9782-1_27. [p1]

P. N. Zivich and A. Breskin. Machine learning for causal inference: On the use of cross-fit estimators. *Epidemiology*, 32(3):393–401, 2021. doi: 10.1097/EDE.0000000000001332. [p6]

Etienne Peyrot
Université Paris Cité and Université Sorbonne Paris Nord, Inserm, INRAE, Center for Research in Epidemiology and StatisticS (CRESS)
F-75004 Paris, France
ORCID: 0009-0006-8520-6201
etienne.peyrot@inserm.fr

François Petit
Université Paris Cité and Université Sorbonne Paris Nord, Inserm, INRAE, Center for Research in Epidemiology and StatisticS (CRESS)
F-75004 Paris, France

https://www.fpetit.org
ORCID: 0000-0003-2258-170X
francois.petit@inserm.fr